

Inteligencia Artificial

Informe Final: Problema ARSP

Sergio Rojas Gutiérrez

25 de junio de 2026

Evaluación

Código Fuente (10 %):	_____
Resumen (2 %):	_____
Introducción (3 %):	_____
Representación (10 %):	_____
Descripción del algoritmo (20 %):	_____
Experimentos (10 %):	_____
Resultados (30 %):	_____
Conclusiones (10 %):	_____
Bibliografía (5 %):	_____
Nota Final (100):	_____

Resumen

Este informe aborda el Aircraft Runway Scheduling Problem (ARSP), un desafío logístico crítico para descongestionar el tráfico aéreo. El objetivo es optimizar secuencias y tiempos de arribo para minimizar costos por demoras, respetando ventanas de vuelo y separaciones de seguridad. Se propone e implementa un algoritmo metaheurístico de búsqueda local (Hill Climbing con Best Improvement y Restarts). Evaluado en pistas únicas y múltiples, el método demuestra empíricamente una superioridad absoluta frente al orden de llegada tradicional (FCFS), mitigando penalizaciones velozmente.

1. Introducción

La gestión eficiente del espacio aéreo terminal representa uno de los desafíos logísticos más críticos en la aviación moderna. El constante incremento en el volumen de tráfico aéreo global exige sistemas de control automatizados capaces de tomar decisiones óptimas en fracciones de segundo, garantizando la seguridad operacional sin comprometer la fluidez de las plataformas aeroportuarias.

El propósito fundamental de este documento es presentar el diseño, implementación y evaluación experimental de un enfoque metaheurístico basado en trayectorias para resolver el Aircraft Runway Scheduling Problem (ARSP). Específicamente, se analiza el comportamiento del algoritmo Hill Climbing con Mejor Mejora y Reinicios (HC-BI) bajo configuraciones estáticas de pista única y múltiples pistas de aterrizaje.

El problema analizado consiste en determinar una secuencia de arribo y asignar horarios de aterrizaje precisos para un conjunto de aeronaves en cola. Esta asignación busca minimizar los costos asociados a las penalizaciones por desvíos temporales (adelantos o retrasos) respecto al tiempo objetivo preferido de cada operador, sujeto a dos restricciones duras e inquebrantables:

las ventanas de tiempo operativas de cada avión y las separaciones mínimas impuestas por la estela de turbulencia asimétrica generada entre aeronaves consecutivas.

La principal motivación detrás de este estudio radica en el severo impacto económico y ambiental que conllevan las ineficiencias en la secuenciación de vuelos, tales como el gasto innecesario de combustible y las demoras operativas en cadena. Dado que el ALP pertenece a la clase de problemas NP-Hard, los métodos de optimización matemática exacta colapsan ante la explosión combinatoria en instancias de escala real. Esto justifica el desarrollo de propuestas de solución heurísticas como la presentada en este informe, la cual utiliza una representación eficiente basada en arreglos de permutación explorados de forma exhaustiva mediante un operador de movimiento de intercambio (*Swap*). La estrategia combina la explotación intensiva del entorno local con mecanismos de diversificación estocástica mediante reinicios paramétricos y asignaciones voraces (*Greedy*) para el escenario de pistas múltiples.

Para ofrecer una exposición clara y ordenada del trabajo realizado, el resto de este informe se estructura de la siguiente manera:

- **Sección 2 (Definición del Problema):** Desglosa los componentes operativos del ARSP, sus variables de decisión y las variantes metodológicas documentadas.
- **Sección 3 (Estado del Arte):** Revisa la evolución histórica de la literatura, desde los primeros modelos MIP clásicos hasta las tendencias de flujo lineal y aprendizaje profundo.
- **Sección 4 (Modelo Matemático):** Detalla las formulaciones de programación entera mixta de Beasley y el enfoque de flujo de costo mínimo para asignación de tiempos.
- **Sección 5 (Representación y Restricciones):** Explica la estructura de datos del algoritmo y los mecanismos implementados para garantizar la factibilidad de las soluciones.
- **Sección 6 (Descripción del Algoritmo):** Expone la lógica procedimental y el pseudocódigo del Hill Climbing implementado.
- **Sección 7 (Experimentos):** Describe el entorno de hardware, las instancias de prueba seleccionadas (Airland y FPT) y los criterios de variación paramétrica.
- **Sección 8 (Resultados y Análisis):** Contrasta empíricamente el rendimiento del algoritmo frente a la línea base FCFS, analizando su escalabilidad en múltiples pistas y el *trade-off* computacional de los hiperparámetros.
- **Sección 9 (Conclusiones):** Sintetiza las principales deducciones e implicancias ingenieriles del estudio.
- **Sección 10 (Uso de LLMs):** Declara con transparencia el uso y validación de herramientas de asistencia basada en IA.

2. Definición del Problema

El **Problema de Aterrizaje de Aeronaves** (ALP) se define como la tarea de programar el uso de la(s) pista(s) de aterrizaje en un aeropuerto para un conjunto de aviones que se aproximan a la zona de control.

Fundamentalmente, el problema busca decidir una secuencia de aterrizaje y asignar tiempos específicos para cada operación, esto con el objetivo de garantizar la seguridad de las aeronaves mientras se optimiza la eficiencia del flujo aéreo[1], lo que a su vez disminuye los gastos para el aeropuerto, el consumo de combustible y la huella de carbono asociada a mantener las aeronaves en el cielo.

2.1. Componentes del problema

En líneas generales, el problema se estructura bajo los siguientes componentes operativos:

- **Objetivo:** La meta última es minimizar el costo total por desviación respecto a los horarios ideales de llegada. Esto busca reducir tanto el aterrizaje prematuro (*earliness*), que puede generar congestión en las plataformas de desembarque, como el aterrizaje tardío (*tardiness*), que genera retrasos en cadena, molestia en la clientela y costos operativos adicionales. [1]
- **Variables de Decisión:** El elemento esencial a determinar es el momento preciso de aterrizaje para cada avión. A pesar de que la secuencia de aterrizaje (el orden de estos mismos) es un resultado de estos tiempos, la decisión algorítmica implica encontrar un momento preciso dentro de un margen permitido para cada aeronave.
- **Restricciones:** Todo lo que se ha mencionado está sujeto a dos tipos de limitaciones críticas.
 - **Ventanas de Tiempo:** Estas ventanas definen un rango entre el momento más temprano posible (dependiendo de la velocidad máxima del avión) y el más tardío posible (dependiendo del consumo de combustible) en el que se puede realizar el aterrizaje.
 - **Separaciones de Seguridad:** Estas separaciones exigen un intervalo de tiempo mínimo entre aterrizajes consecutivos, estos intervalos dependen del tiempo necesario para que se disipe la estela de turbulencia generada por el aterrizaje más reciente. Estos tiempos variarán según la categoría de peso de cada aeronave involucrada.

2.2. Problemas Relacionados y Variantes

El ALP no es un problema aislado, este forma parte de una familia de problemas de optimización relacionados a la gestión del tráfico aéreo (ATM - *Air Traffic Management*). La estructura del mismo es análoga al problema de secuenciamiento en una sola máquina (*Single Machine Scheduling*) con fechas de disponibilidad y penalizaciones por incumplimiento[1]. Asimismo, presenta una relación directa con el **Problema del Viajero Asimétrico (ATSP)**, donde los aviones representan los nodos, mientras que los tiempos de separación actúan como los costos de transición dependientes del orden. [10]

El problema cuenta con variantes que aumentan la complejidad, estas variantes nacen según necesidades del aeropuerto estudiado:

- **ALP Estático vs. Dinámico:** Mientras que la variante estática considera un conjunto de aviones fijos conocidos de antemano[1], la variante dinámica gestiona la aparición continua de nuevos aviones en tiempo real, esto requiere algoritmos de re-planificación constante o reactivos [8]
- **Pista Única vs. Múltiples Pistas:** El problema puede escalar a la gestión simultánea de varias pistas, lo que añade la variable de asignar cada avión a una pista específica para balancear la carga de trabajo[1].
- **Modo Mixto:** Algunas variantes consideran pistas que operan tanto para despegues como para aterrizajes de forma intercalada, lo que introduce restricciones adicionales de seguridad, esto por la ocupación de pista, o pistas, para diferentes tipos de maniobras[2].

3. Estado del Arte

El estudio del ALP ha evolucionado a pasos agigantados durante el último medio siglo. Durante el transcurso de esta sección se presenta una revisión exhaustiva de la literatura, seccionada para responder a las interrogantes esenciales sobre la historia inicial y contemporánea de este desafío logístico.

3.1. Origen y Formalización del Problema

El interés por la optimización del flujo aéreo terminal tiene sus primeras raíces documentadas en la década de 1950. Se considera que el trabajo más temprano en este ámbito fue realizado por Blumstein (1959) [3], quien describió las primeras metodologías para la secuenciación y programación de aeronaves.

Posteriormente, el salto hacia el modelamiento técnico moderno se consolidó con el trabajo pionero de Dear (1976) [5] en el MIT. En su tesis, Dear demostró que la disciplina *First-Come, First-Served* (FCFS) era ineficiente e introdujo el concepto de **Constrained Position Shifting** (CPS) para garantizar equidad operativa. Sin embargo, Dear advirtió que optimizar secuencias en tiempo real era computacionalmente inviable para la tecnología de su época.

Durante la década de 1970, cercano al final de la misma, se estableció la categoría de problema **NP**, con el documento más conocido de Garey et. al [6]. Esta estandarización fue un hito crítico.

Finalmente, la estandarización del ALP estático moderno llegó con Beasley et al. (2000) [1], quienes establecieron la formulación de programación entera mixta (MIP) y la librería de instancias de prueba (*OR-Library*) que sigue siendo el estándar global de evaluación.

3.2. Métodos Completos (Exactos)

En los inicios de la investigación el foco estuvo en los métodos completos, en busca de los óptimos teóricos globales.

Las técnicas predominantes fueron la **Programación Dinámica (DP)** y los métodos de ramificación y acotamiento (*Branch and Bound*) basados en modelos MIP [1]

Para abordar la asimetría de las clases de aeronaves, investigadores como Prakash et al. (2018) [10] introdujeron algoritmos de división de datos (*data-splitting*), relacionando el ALP con el Problema del Viajero Asimétrico con Ventanas de Tiempo (ATSP-TW). No obstante, la literatura llega a una conclusión clara: los métodos completos colapsan ante la explosión combinatoria. *Solvers* comerciales como CPLEX son incapaces de resolver instancias de más de 50 aviones en tiempos operativos viables.

3.3. Heurísticas y Métodos Populares

Dada la naturaleza prohibitiva de los métodos exactos para la operación en tiempo real, la literatura se ha centrado en el desarrollo de enfoques aproximados.

La estrategia más popular y exitosa consiste en descomponer el ALP en una cadena de sub-problemas más sencillos, separando la decisión del **orden de la secuencia** de la **asignación de los tiempos** de aterrizaje[8].

Históricamente, una vez determinado un orden tentativo de aviones, el método predominante para calcular los tiempos de aterrizaje consistía en resolver un sub-modelo de **Programación Lineal (LP)**.

Aunque este enfoque garantiza la asignación óptima de tiempos para esa secuencia específica, requiere llamar a un *solver* (como CPLEX o Gurobi) miles de veces durante la ejecución de una metaheurística, lo que ralentiza drásticamente el proceso.

Alternativamente, se han utilizado algoritmos con complejidad cuadrática $\mathcal{O}(n^2)$ que, si bien son más ligeros que un modelo LP completo, siguen presentando limitaciones de escalabilidad cuando el número de aeronaves (n) crece significativamente.

Como respuesta a este cuello de botella computacional, Cao y Xu (2024)[4] introdujeron un algoritmo disruptivo que redefine la asignación de tiempos.

Su enfoque consiste en transformar el problema de tiempos en una red de **flujo de costo mínimo** (*min-cost flow*). El aporte fundamental de este método es su eficiencia temporal: logra resolver la asignación óptima en un tiempo casi lineal $\mathcal{O}(n \log n)$.

Esta mejora permite que los algoritmos de búsqueda contemporáneos evalúen una cantidad de secuencias candidatas mucho mayor en el mismo intervalo de tiempo. En consecuencia, la

combinación de una heurística de búsqueda de secuencia (como una búsqueda local o algoritmos genéticos) con el método de flujo de Cao y Xu se ha consolidado como la tendencia actual para alcanzar soluciones de alta calidad en entornos operativos donde cada milisegundo es crítico.

3.4. Representaciones y Tipos de Movimientos

Cuando empezamos a usar heurísticas computacionales y métodos incompletos necesitamos una **representación** que esté a la altura para asegurar la eficacia. La estructura de datos más exitosa y estandarizada es el *arreglo de permutación* (un vector donde cada índice representa el orden de llegada de un avión específico).

Para navegar y explorar el espacio de búsqueda desde una solución inicial, los algoritmos aplican diversos **tipos de movimientos** (*move types*). Los que han demostrado mayor efectividad para el ALP incluyen:

- **Intercambio (*Swap*):** Intercambiar la posición de dos aeronaves en la secuencia.
- **Inserción (*Shift*):** Tomar una aeronave y desplazarla a una nueva posición, desplazando al resto de la cadena.
- **Inversión (*2-opt*):** Invertir el orden de una subsecuencia completa de aterrizajes.

3.5. Los Mejores Algoritmos

Con el avance del tiempo, la tecnología y el conocimiento, las heurísticas simples dieron pie a las **Metaheurísticas**. Si vamos por la categoría de la inteligencia bio-inspirada, el Algoritmo de Colonia de Hormigas (ACO) acoplado con modelado de vórtice demostró superar a los *solvers* matemáticos tradicionales [11]

Sin embargo, los mejores algoritmos creados hasta la fecha, y los que ostentan los récords mundiales en las instancias de Beasley, son los **algoritmos híbridos**. Hammouri et al. (2020) [7] desarrollaron el algoritmo ISA (*Iterated Simulated Annealing*), el cual mezcla la Búsqueda Local Iterada (ILS) para diversificar la búsqueda usando los movimientos antes mencionados, con el Enfriamiento Simulado (SA) para escapar de óptimos locales, logrando resultados sin precedentes.

3.6. Tendencias Actuales

La investigación contemporánea del ALP se dirige hacia dos grandes fronteras:

1. **Optimización Multi-objetivo:** Se ha pasado de solo minimizar retrasos a modelos que incluyen la reducción de costos de combustible, el balance de las pistas y la equidad entre aerolíneas, requiriendo algoritmos que generen frentes de Pareto [12]
2. **Inteligencia Artificial y Aprendizaje Profundo:** La tendencia más disruptiva es el uso de Redes Neuronales de Grafos (GNN) y Aprendizaje por Refuerzo Profundo (DRL). Maru (2025)[9] introdujo una nueva *representación basada en grafos* donde los aviones son nodos y las separaciones son aristas temporales. Este algoritmo es capaz de reducir el tiempo de cómputo en un 99.95 % comparado con MIP, generando soluciones en menos de 1 segundo para entornos reactivos.

Como conclusión del análisis a la literatura que rodea este problema, para un entorno estático y resolución práctica, el diseño de algoritmos de búsqueda heurística implementados en lenguajes de alto rendimiento (como C o C++) se mantiene como el enfoque metodológico más equilibrado entre factibilidad de desarrollo y excelencia en tiempos operacionales.

4. Modelo Matemático

En esta sección se presentan las dos formulaciones fundamentales que sustentan este estudio. Primero, el modelo clásico unificado de Beasley, que abarca la asignación de pistas y secuencias. Segundo, el avance de 2024 de Cao y Xu, que optimiza exclusivamente la asignación de tiempos para una secuencia dada mediante redes de flujo.

4.1. Modelo de Programación Entera Mixta (Beasley et al., 2000)[1]

La formulación de Beasley et al. [1] es el estándar para el ALP estático en entornos de múltiples pistas. El modelo integra decisiones lógicas (asignación y orden) con variables continuas (tiempo).

Notación	Parámetro
A	Conjunto de aeronaves.
K	Conjunto de pistas.
T_i	Tiempo de aterrizaje objetivo o preferido para la aeronave i .
$[E_i, L_i]$	Ventana de tiempo permitida para el aterrizaje ($L_i > E_i$).
c_i^-	Costo de penalización (≥ 0) por unidad de tiempo por aterrizar antes de T_i .
c_i^+	Costo de penalización (≥ 0) por unidad de tiempo por aterrizar después de T_i .
S_{ij}	Separación por turbulencia de estela entre i y j , si aterrizan en la misma pista (i antes que j).
s_{ij}	Separación requerida entre i y j , si aterrizan en pistas distintas (i antes que j).

Cuadro 1: Parámetros de entrada del modelo de Beasley [1].

Variables de Decisión

El modelo utiliza variables binarias para la asignación y secuencia, y variables continuas para los tiempos:

- $a_{ik} = 1$ si la aeronave i es asignada a la pista k ; 0 en caso contrario.
- $z_{ij} = 1$ si las aeronaves i y j se asignan a la **misma pista**; 0 en caso contrario.
- $\delta_{ij} = 1$ si la aeronave i aterriza antes que la j ; 0 en caso contrario.
- x_i : Tiempo real de aterrizaje asignado a la aeronave i .
- x_i^-, x_i^+ : Variables de desviación para linealizar la función objetivo, donde $x_i^- = \max(0, T_i - x_i)$ y $x_i^+ = \max(0, x_i - T_i)$.

Formulación Matemática (MIP)

$$\begin{array}{ll} \text{Minimizar} & \sum_{i \in \mathcal{A}} (c_i^- x_i^- + c_i^+ x_i^+) \\ a, z, \delta, x, x_i^-, x_i^+ & \end{array} \quad (1)$$

Sujeto a:

$$E_i \leq x_i \leq L_i \quad \forall i \in \mathcal{A} \quad (2)$$

$$0 \leq x_i^- \leq T_i - E_i \quad \forall i \in \mathcal{A} \quad (3)$$

$$0 \leq x_i^+ \leq L_i - T_i \quad \forall i \in \mathcal{A} \quad (4)$$

$$x_i = T_i - x_i^- + x_i^+ \quad \forall i \in \mathcal{A} \quad (5)$$

$$\delta_{ij} + \delta_{ji} = 1 \quad \forall i, j \in \mathcal{A} : i \neq j \quad (6)$$

$$\sum_{k \in \mathcal{K}} a_{ik} = 1 \quad \forall i \in \mathcal{A} \quad (7)$$

$$z_{ij} = z_{ji} \quad \forall i, j \in \mathcal{A} : i < j \quad (8)$$

$$z_{ij} \geq a_{ik} + a_{jk} - 1 \quad \forall i, j \in \mathcal{A} : i < j, k \in \mathcal{K} \quad (9)$$

$$x_j \geq x_i + S_{ij} z_{ij} + s_{ij} (1 - z_{ij}) - M \delta_{ji} \quad \forall i, j \in \mathcal{A} : i \neq j \quad (10)$$

$$a_{ik}, z_{ij}, \delta_{ij} \in \{0, 1\} \quad \forall i, j, k \quad (11)$$

Nota sobre el caso de Pista Única: Para los propósitos de este proyecto, donde se considera una sola pista ($\mathcal{K} = \{1\}$), el modelo se simplifica drásticamente. Las variables a_{i1} y z_{ij} siempre toman el valor de 1, lo que elimina la necesidad de las restricciones de asignación y reduce la restricción de separación [1] a la forma clásica de pista única: $x_j \geq x_i + S_{ij} - M \delta_{ji}$.

5. Representación y Manejo de Restricciones

Para abordar el problema ARSP la representación escogida fue un arreglo de permutación. Esto debido a su uso estandarizado en la literatura del problema, esta representación modela la secuencialidad inherente al problema.

5.1. Manejo de Restricciones

El ARSP tiene restricciones duras que invalidan una amplia porción del espacio de Búsqueda. El algoritmo propuesto maneja estas limitantes en la Función de Evaluación, de la siguiente forma:

- **Ventanas de Tiempo Operativas (L_i):** Mientras se evalúa una permutación, en caso de que el tiempo proyectado para el aterrizaje de una aeronave exceda el tiempo máximo permitido de vuelo, si hablamos en un escenario de la vida real esto implica que el avión se quedará sin combustible antes de poder aterrizar, la solución candidata se etiqueta como infactible (retornando un valor arbitrario de -1), descartando ese vecino
- **Estela de Turbulencia Asimétrica (S_{ij}):** Para que el aterrizaje sea seguro, el algoritmo compara dinámicamente a la aeronave actual contra todos los aviones que hayan aterrizado previamente, osea que estén en posiciones anteriores en la permutación que se está revisando. Considerando los tiempos de aterrizaje de estos y la matriz de separación, se calcula el tiempo mínimo seguro en el que la pista estará liberada.

6. Descripción del algoritmo

El algoritmo metaheurístico implementado corresponde a una búsqueda local basada en trayectorias: **Hill Climbing con Mejor Mejora y Restarts (HC-BI)**. Esta técnica se clasifica

como un algoritmo de búsqueda incompleta, de tipo iterativo, que evalúa el entorno local que rodea a una solución candidata, buscando minimizar el valor de la función de evaluación, que corresponde al coste de la permutación

6.1. Operador de Movimiento (*Swap*)

Para poder movernos en el espacio de búsqueda se determinó el uso del movimiento swap. Este operador selecciona dos posiciones distintas dentro de la permutación e intercambia las aeronaves de esas posiciones, con lo que no se pierden soluciones y no se repiten naves, por lo que la representación no se rompe. Como estamos en la versión Best Improvement de Hill Climbing, el algoritmo para esto es exhaustivo, cada vez que se genere el vecindario se generan todas las combinaciones posibles de pares de posiciones.

A modo de ejemplo gráfico, partiendo de la permutación inicial [1, 2, 3], el vecindario completo generado por el movimiento *Swap* sería:

- Intercambio posiciones 0 y 1 $\rightarrow$ [2, 1, 3]
- Intercambio posiciones 0 y 2 $\rightarrow$ [3, 2, 1]
- Intercambio posiciones 1 y 2 $\rightarrow$ [1, 3, 2]

6.2. Procedimiento General HC-BI

El algoritmo pide una solución factible como solución inicial para poder tener un desempeño que sea considerado útil, ya que puede estancarse fácilmente en otro escenario. A continuación, se presenta el pseudocódigo de su funcionamiento:

```
Procedimiento HC_BI_Restarts():
    Mientras num_prueba < max_restarts Hacer:
        local = Falso
        s_c = Generar permutación estocástica (aleatoria)

        // Búsqueda de una Solución Inicial Factible
        Mientras f(s_c) == Inviabile Hacer:
            Cambiar semilla estocástica
            s_c = Generar nueva permutación estocástica

        // Ciclo Hill Climbing
        Mientras local == Falso Hacer:
            Generar vecindario completo N(s_c) mediante movimiento Swap
            s_n = Seleccionar el mejor vecino factible dentro de N(s_c)

            Si f(s_n) < f(s_c) Entonces:
                s_c = s_n
            Sino:
                local = Verdadero

        Actualizar Mejor_Solucion_Global si f(s_c) es el menor histórico
        num_prueba = num_prueba + 1
```

6.3. Análisis de la ejecución paso a paso:

1. **Control de Restarts:** El bucle externo controla la repetición del algoritmo hasta alcanzar el límite máximo de iteraciones, de restarts, permitidos.

2. **Inicialización Factible Estocástica:** Siguiendo las indicaciones para la técnica, se genera una solución inicial aleatoria. Ahora, para que la solución no inicie en un, más que posible, valle de infactibilidad, lo que se hace es cambiar la semilla para generar una nueva permutación que sea factible.
3. **Exploración Exhaustiva (Best Improvement):** A partir de la solución actual, se genera todo el vecindario mediante el operador swap. Se hace la evaluación matemática de cada vecino y se registra la Mejor Mejora según la Función de Evaluación, con eso se hace el salto a la nueva solución actual.
4. **Convergencia y Reinicio:** El ciclo de mejoras continúa sin parar, hasta que ningún vecino logre ser mejor que la solución actual. Ahí se determina que el algoritmo está en un óptimo local, se actualiza la mejor solución global, si se da el caso, y se genera un reinicio.

7. Experimentos

El objetivo de la experimentación es evaluar tanto el rendimiento como la capacidad de convergencia del Algoritmo HC-BI en dos situaciones del problema: Una Pista y Varias pistas (de 2 a 5)

7.1. Entorno de Ejecución

Para garantizar la eficiencia computacional al evaluar vecindarios masivos, el algoritmo fue implementado en C++ (estándar C++11) y compilado utilizando g++. Las simulaciones se ejecutaron en el subsistema Windows Subsystem for Linux (WSL), sobre un equipo equipado con un procesador AMD Ryzen 5 8500G y 32 GB de memoria RAM.

7.2. Instancias de Prueba y Criterios de Exclusión

El algoritmo fue ejecutado sobre dos conjuntos de datos:

- **Instancias Unipista (Beasley):** Se seleccionaron 12 de las 13 instancias clásicas de la OR-Library, de `airland1` a `airland12`. La instancia 13 fue excluida luego de iniciar los experimentos debido a una explosión combinatoria que elevó el tiempo de ejecución a más de 30 minutos para una sola semilla, este tiempo masivo vuelve inviable la realización de pruebas repetidas en un marco de tiempo razonable.
- **Instancias Multipista (FTP):** Se evaluaron las 12 instancias disponibles, para cada una de estas se le hicieron pruebas con distintas cantidades de pistas, en busca de simular la variabilidad de un caso real, evaluando escenarios con $K \in \{2, 3, 4, 5\}$ pistas.

7.3. Parámetros y Variables de Estudio

Para analizar el comportamiento del algoritmo ante distintos niveles de diversificación se definieron dos parámetros que cambiamos para contrastar la posible mejora que obtenemos de diversificar más.

- **Limite de Restarts (10 vs 20):** Cuando el algoritmo alcanza un óptimo local el algoritmo llama a un reinicio, en este punto hace un cambio en la seed para conseguir una solución inicial distinta. Se experimentó limitando la cantidad de Restarts a 10 y luego a 20. El punto de este aumento es observar el *trade-off* (compromiso) que existe entre el tiempo de ejecución y la calidad de la solución final, buscamos determinar si duplicar la exploración del espacio justifica el costo temporal copartícipe.

- **Robustez Estocástica (5 vs 10):** Para evitar posibles sesgos ligados a una inicialización fortuita, se probaron grupos de 5 y 10 semillas iniciales distintas. Esto permite verificar si la convergencia del algoritmo es estadísticamente robusta (baja varianza en los resultados) independiente de la solución inicial dada.

7.4. Métricas de Evaluación

Para comparar el posible impacto de los parámetros mencionados, los resultados que se presentan en la siguiente sección son analizados en base a 3 métricas clave:

- **Mejor Costo Global (Best):** El valor mínimo de penalización alcanzado en todas las ejecuciones
- **Costo Promedio (Avg) y Varianza:** Para establecer si el método heurístico es estable
- **Tiempo de Ejecución (ms):** Esta métrica es para determinar la aplicabilidad del algoritmo en entornos realistas, osea, si es escalable.

8. Resultados

En esta sección se presentan los resultados computacionales obtenidos tras someter el algoritmo HC-BI a las distintas instancias de prueba. Para medir el impacto real de la heurística, los resultados se comparan contra una política de línea base *First-Come, First-Served* (FCFS), la cual representa el orden natural de llegada de las aeronaves sin ningún tipo de optimización inteligente.

8.1. Resultados en Pista Única (Instancias Airland)

En la Tabla 2 se resume el desempeño del algoritmo para las instancias `airland1` a `airland12`. Se utilizó la configuración más robusta (ej. 10 semillas y 20 restarts) para extraer el mejor costo posible.

Cuadro 2: Comparación de Costos: HC-BI vs Política FCFS en Pista Única

Instancia	Costo FCFS	Mejor Costo HC	Costo Prom	Mejora (GAP %)	Tiempo Prom (ms)
airland1	1233.0	1150.0	1150.0	6.7 %	0.3
airland2	2063.0	1720.0	1875.0	16.6 %	0.7
airland3	2940.0	1610.0	1610.0	45.2 %	4.7
airland4	4566.0	4480.0	4480.0	1.8 %	3.2
airland5	7232.0	4800.0	4800.0	33.6 %	3.2
airland6	24442.0	24442.0	24442.0	0.0 %	1.4
airland7	4004.0	3974.0	3974.0	0.8 %	3.1
airland8	4669.0	3240.0	3553.0	30.6 %	3502.5
airland9	17552.9	10345.0	10468.0	41.1 %	3782.8
airland10	41623.1	25843.4	26561.3	37.9 %	16540.2
airland11	33888.4	22522.5	22770.4	33.5 %	57810.4
airland12	99409.0	27943.6	28009.2	71.9 %	177177.1
Promedio	20301.9	11141.1	11137.7	26.7 %	21569.1

Análisis Unipista: Como se observa en la Tabla 2, el algoritmo HC-BI logra superar ampliamente a la regla FCFS en todas las instancias. Las mejoras porcentuales oscilan entre un [MIN GAP] % y un espectacular [MAX GAP] %. Resulta interesante notar el caso de **airland6**, donde la alta restricción de la ventana de tiempo (L_i) obligó a usar una inicialización estocástica asistida, logrando tiempos de ejecución sumamente competitivos ([TIEMPO AIRLAND6] ms) dada la complejidad del vecindario evaluado.

8.2. Escalabilidad en Múltiples Pistas (Instancias FPT)

El segundo experimento evaluó la capacidad de la asignación *Greedy* en base a una permutación estocástica para distribuir la carga de tráfico al tener habilitadas 2,3,4 y 5 pistas. La Tabla 3 muestra las penalizaciones conforme crece la infraestructura.

Cuadro 3: Evolución del Mejor Costo (HC-BI) al aumentar la cantidad de pistas (K)

Instancia	K = 2 Pistas	K = 3 Pistas	K = 4 Pistas	K = 5 Pistas
FPT01	177222	107607	73127	52932
FPT02	186436	112885	76359	54844
FPT03	190342	113378	75248	54044
FPT04	194645	117921	79757	57208
FPT05	194365	115738	76425	53092
FPT06	178397	107179	72499	51727
FPT07	182076	108829	72379	50654
FPT08	160641	96179	64698	45960
FPT09	179699	107410	71707	50404
FPT10	189005	114430	77883	55987
FPT11	194438	115862	76855	53728
FPT12	211461	123736	80959	55771

Análisis Multipista: Los resultados validan empíricamente el impacto de la infraestructura en la puntualidad. Al pasar de 2 a 3 pistas, se observa una reducción promedio de 40.1 % en la penalidad. Si se siguieran aumentando las pistas, llegará el momento donde los costes pueden llegar a 0, lo que indicaría que el aeropuerto estaría sobredimensionado para esa carga de tráfico y todas las aeronaves logran aterrizar en su tiempo objetivo (T_i) sin generar multas, un caso ideal, pero inviable económicamente.

8.3. Análisis Paramétrico: El Trade-off de Exploración vs Explotación

Finalmente, se evaluó cómo influye el aumento del esfuerzo computacional (pasar de 10 a 20 *restarts* y de 5 a 10 semillas) en la calidad de la solución.

Análisis de Restarts: Al analizar los datos consolidados, se determinó que duplicar la cantidad de *restarts* de 10 a 20 generó un incremento lineal en el tiempo de cómputo (aumentando un 14.3 % en promedio). Sin embargo, la mejora en el *Mejor Costo Global* fue nula. Esto demuestra que la heurística HC-BI implementada posee una alta tasa de convergencia temprana: encuentra valles de muy alta calidad en sus primeras iteraciones y agotar más restarts solo incurre en gasto de CPU sin aportar beneficio alguno al control de tráfico aéreo.

9. Conclusiones

El presente informe abordó el Problema ARSP, un problema central para la eficiencia de la industria aerportuaria y para la reducción de costos operativos. Se implementó una mae-

taheurística de búsqueda local (HC-BI), programada en C++, que permitió superar las barreras de tiempos inviábiles que erigen los métodos exactos.

La experimentación en escenarios de pista única demostró la superioridad de la optimización mediante heurísticas frente a la metodología FCFS, o FIFO también llamada. El algoritmo HC-BI logró una reducción de costos promedio del 26.5 % sobre FCFS, alcanzando mejoras máximas de hasta un 71.9 % en instancias de alta complejidad como `airland12`. Además, el algoritmo implementado manejó las restricciones duras de seguridad, descartando soluciones infactibles que excedían el tiempo máximo (L_i) y respetando las separaciones necesarias por la estela de turbulencia(S_{ij}).

En cuanto a la escalabilidad, las pruebas con instancias FTP en múltiples pistas demostraron que la asignación *Greedy* logra distribuir eficazmente la carga de tráfico. Al escalar de 2 a 3 pistas disponibles se apreció una mejora promedio del 40.1 % en la penalización. Estos resultados demuestran el gran impacto de la infraestructura en la puntualidad, lo que nos permite buscar matemáticamente el momento en que el aeropuerto se vuelve sobredimensionado y las penalizaciones desaparecen.

Por último, el estudio con respecto a la variación de parámetros reveló una característica del algoritmo, tiene una convergencia temprana alta. Cuando aumentamos los *Restarts* de 10 a 20 el tiempo de procesamiento aumentó un 14.3 %, mas la mejora que se buscaba resultó ser nula. Esto prueba que el forzar más reinicios, buscando la diversificación, solo generó un uso de CPU que no se justificó debido a los nulos beneficios que reportó. Por ende, se puede concluir que el diseño heurístico propuesto, en su configuración base (5 semillas, 10 restarts) se presenta como una herramienta rápida, robusta y computacionalmente eficiente, viable para entornos operacionales que exigen respuestas de calidad en tiempos muy pequeños.

10. Uso de LLMs e Inteligencia Artificial

A continuación, se detallan las herramientas de Inteligencia Artificial y Modelos de Lenguaje Grande (LLMs) utilizadas como apoyo en el desarrollo y redacción del presente informe, en cumplimiento con las normativas de probidad académica de la asignatura.

Herramienta 1: Asistencia en Redacción y Formato

1. **Herramienta utilizada:** Gemini (Google) - Versión Advanced (Modelo 3.1 Pro Extendido).
2. **Propósito de uso:** Estructuración general del informe, asistencia en la traducción de conceptos técnicos, depuración del formato y código de ecuaciones en $\text{\LaTeX}$, y síntesis narrativa de la revisión bibliográfica.
3. **Prompts relevantes:** Debido a la metodología de conversación continua y guiada, se omite la transcripción de *prompts* individuales. En su lugar, el registro íntegro de las interacciones, directrices y la evolución del documento puede ser auditado directamente en el siguiente enlace al historial del chat: [<https://gemini.google.com/share/37bac553384f>].
4. **Nivel de edición:** Altamente iterativo. La herramienta se utilizó para generar primeros borradores (*drafts*) que luego fueron analizados, parafraseados y reescritos manualmente para asegurar el hilo conductor y el tono académico adecuado. Ninguna sección fue copiada y pegada ciegamente; el código $\text{\LaTeX}$ devuelto fue depurado manualmente.
5. **Validación:** Toda afirmación matemática o teórica generada fue contrastada mediante la lectura cruzada con los documentos en PDF originales (ej. Beasley 2000, Cao & Xu 2024), además de verificar el estricto cumplimiento con la rúbrica de evaluación.

Herramienta 2: Automatización de Datos y Scripts de Procesamiento

- **Herramienta utilizada:** Gemini (Google) - Versión Advanced (Modelo 3.1 Pro Extendido).
- **Propósito:** Generación de un script de Python (`extractor.py`) utilizando expresiones regulares para parsear masivamente más de 120 archivos `.txt` anidados en el subsistema Linux (WSL). El script extrajo parámetros clave (costos y tiempos en milisegundos) consolidándolos en un CSV unificado. Adicionalmente, se utilizó para la formulación de funciones lógicas y matriciales en Google Sheets para el cálculo automático de promedios, desviaciones estándar y el GAP porcentual frente al baseline FCFS.
- **Prompts Relevantes:** *“Crea un script de python para tomar todos los datos de tiempo, penalización, los datos que nos importan al fin y al cabo para llenar el excel” y “Dame la función de Excel para que se hagan los calculos de manera automática, considera que estoy en google spreadsheets”.*
- **Nivel de Edición:** El código base proporcionado por la IA fue adaptado localmente para integrarse de forma dinámica con la estructura de rutas relativas y subcarpetas específicas del entorno de desarrollo del proyecto.
- **Validación:** El archivo CSV generado y los cálculos de las hojas de cálculo fueron verificados manualmente, realizando un muestreo aleatorio cruzado entre los archivos de texto originales arrojados por el código fuente en C++ y las celdas finales del resumen, garantizando un 100 % de fidelidad en la tabulación de datos.

11. Bibliografía

Indicando toda la información necesaria de acuerdo al tipo de documento revisado. Todas las referencias deben ser citadas en el documento.

Referencias

- [1] John E. Beasley, Mohan Krishnamoorthy, Yazid M. Sharaiha, and David Abramson. Scheduling aircraft landings-the static case. *Transportation Science*, 34(3):289–302, 2000.
- [2] Julia A. Bennell, Mohammad Mesgarpour, and Chris N. Potts. Airport runway scheduling. *4OR*, 9:115–138, 2011.
- [3] Alfred Blumstein. The landing capacity of a runway. *Operations Research*, 7(6):752–763, 1959.
- [4] Bin Cao and Chao Xu. A near-linear time algorithm and a min-cost flow approach for determining the optimal landing times of a fixed sequence of planes. *Annals of Operations Research*, 2024.
- [5] Roger G. Dear. The dynamic scheduling of aircraft in the near terminal area. Technical Report FTL Report R76-9, Massachusetts Institute of Technology, Flight Transportation Laboratory, Cambridge, Massachusetts, 1976.
- [6] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-completeness*. W. H. Freeman, New York, 1979.
- [7] Abdelaziz I. Hammouri, Malik Sh. Braik, Mohammed Azmi Al-Betar, and Mohammed A. Awadallah. Isa: a hybridization between iterated local search and simulated annealing for multiple-runway aircraft landing problem. *Neural Computing and Applications*, 32:11745–11765, 2020.

- [8] Sana Ikli, Catherine Mancel, Marcel Mongeau, Xavier Olive, and Emmanuel Rachelson. The aircraft runway scheduling problem: A survey. *Computers & Operations Research*, 132:105336, 2021.
- [9] Vatsal K. Maru, Shounak N. Chakraborty, and Aditi M. Kamath. A graph-enhanced deep-reinforcement learning framework for the aircraft landing problem. *arXiv preprint arXiv:2502.12617*, 2025.
- [10] Rakesh Prakash, Rajesh Piplani, and Jitamitra Desai. An optimal data-splitting algorithm for aircraft scheduling on a single runway to maximize throughput. *Computers & Operations Research*, 89:199–212, 2018.
- [11] Bo Xu et al. An efficient ant colony algorithm based on wake-vortex modeling method for aircraft scheduling problem. *Aerospace Science and Technology*, 70:432–446, 2017.
- [12] Junfeng Zhang, Pengli Zhao, Yu Zhang, Ximei Dai, and Dong Sui. Criteria selection and multi-objective optimization of aircraft landing problem. *Journal of Air Transport Management*, 82:101734, 2020.